YOUR GRADUATION TASK

從自己的工作中，找出值得交給 Agent 的事

從真實、而且會反覆發生的工作開始。**先確認它值得自動化，再設計怎麼交辦、怎麼驗收、例外怎麼處理。**

每週的主題情報整理

現在的問題　要固定追同一批主題，資料散在各處，每週都重做一次，而且怕漏掉。

適合交給它　固定去同幾個來源收資料、去掉重複的、照同一個格式整理成一份。

你要留著的判斷　哪幾條值得放進來、來源可不可信、最後的結論怎麼下。

網站的品質檢查

現在的問題　改完一個地方，不確定有沒有弄壞別的，每次都要自己點一遍。

適合交給它　照規格把檢查跑一遍，列出哪幾條沒過、是在哪個檔案。

你要留著的判斷　要不要照它說的改，以及重要的變更要不要上線。

每個月的營運報表

現在的問題　每個月彙整同一批資料，步驟固定但很花時間，錯一個欄位整份就錯。

適合交給它　讀檔案、核對格式與筆數、跟上個月比、產出同一種報表。

你要留著的判斷　資料本身有問題的時候怎麼處理，以及數字背後要怎麼解讀。

一直在更新的知識庫

現在的問題　文件散在各處、版本對不起來，新的進來不知道跟舊的哪裡衝突。

適合交給它　比對新舊文件，標出重複、衝突與過期的段落，產出更新草稿。

你要留著的判斷　兩份打架的時候哪一份算數，以及什麼可以併進正式版。

同一個需求的三種做法

單次　把資料丟進對話框問一次，做完就結束。下個月同一件事，規格、格式與例外要再講一遍。

重複　請它寫一支程式，你自己收著、每次自己跑。資料換了、來源改了，你自己改那支程式。

系統化　規格、驗收條件、進度與失敗處理都存下來，而且確認它每一輪真的讀得到、照著做。

上面那四件工作，用對話框都做得完一次。**下個月再做同一件，你還要重新交代哪些東西？**

你會重講的那幾句，就是要寫下來的規格與驗收條件。 挑會再發生的那一種工作，四個案例挑一項就好；只做一次的事， 先比排流程的時間跟省下來的時間哪個多。

挑好之後，第一句話貼這個　第一輪只要它分析，不要它動手　點一下複製

```
我想把一件工作交給 Agent。先不要動手，也不要安裝任何工具。

這件工作是：（一句話寫清楚）
現在的做法是：（誰做、用什麼工具、大概幾步）
多久發生一次、每次大概多少量：（例如每個月一次、每次兩百筆）
我要的產出是：（做完我手上會拿到什麼）
目前最花時間、或最容易出錯的一步是：（哪一步）

請先回答這五件事：
1. 把這件工作拆成步驟，標出哪幾步是固定的、哪幾步需要人判斷。
2. 這件事比較適合哪一種：傳統自動化、AI 助手、Workflow，還是 Agent？說明你的理由。
3. 會用到哪些資料、需要哪些權限、有哪些風險。
4. 每一步怎麼知道做對了？驗收條件可以怎麼寫。
5. 如果只先試一小塊，你建議從哪裡開始？

有哪些你判斷不了、需要我補的資訊，直接列出來，不要自己假設。
用清單回答，先不要寫程式。我確認過這份再一起把流程設計出來。
```

VALIDATE WITH MOCK DATA

正式資料還沒到手，先用模擬資料驗證流程

正式資料還沒到手、要等權限，或是裡面有客戶與個人資料的時候，**先用模擬資料把流程驗一輪**： 欄位用一樣的設計，值自己編。正式資料能不能用，同一時間去問。

手上本來就有一份可以用的測試資料，那就直接用那一份，這一段跳過。

模擬資料跑得出這三件事，流程就算站得住

欄位讀對了嗎
它認得哪一欄是日期、哪一欄是金額嗎？

算出來對不對
合計跟你人工核對的答案一樣嗎？

缺資料怎麼辦
少一筆日期的時候，它會停下來問，還是自己補一個？

模擬資料要跟正式那一份同一套欄位與格式，而且要放幾筆真的會遇到的狀況： 缺欄位、日期格式不一樣、金額是零或負數。**每一筆都正常的資料驗不出什麼**。

問之前，先把資料的去向講清楚 哪一份資料 → 去哪個平台 → 產出放哪裡

問句　問你的主管、資訊單位，或是簽約的那個窗口　　點一下複製

我想用【哪個工具】做【哪一件工作】，已經用模擬資料跑過一輪流程。
正式的那一份會用到【哪些欄位】，資料會送到【哪個平台】，產出存在【哪裡】、給【誰】看。
請問這樣可以嗎？如果不行，哪些欄位要先去掉或換掉？
這件事如果要別人決定，可以告訴我找誰嗎？

回覆通常不是「可以」或「不行」兩種，多半是「哪些欄位可以、要用哪個帳號、存在哪裡」。 那幾條回來之後寫進規格文件，下一輪照著跑。

模擬資料跑得過，代表流程站得住，不代表可以上線： 正式資料的欄位、量體與權限都要再驗一次。

YOUR FIRST AGENT WORKFLOW

把工作拆成可交辦、可驗收的流程

交辦之前把範圍、產出與驗收條件寫下來，做完之後真的去對一遍。選一個範圍小、風險低的任務，把一次交辦、一次驗收，加上第二次執行走完。

01 建立工作環境

DEFINE 之前

開一個專案資料夾，準備一份安全的測試資料，確認這一輪會用到哪些工具、它有哪些權限。

```
$ mkdir my-project && cd my-project
$ claude
```

02 確認規格與執行計畫

DEFINE · PLAN

請它先整理出工作範圍、輸入、產出、限制與驗收條件，缺的資料列成待確認問題，先不要動手。你核准之後才開始執行。

03 執行與驗收

EXECUTE · VERIFY

從最小的那一塊開始，自己檢查實際成果，記下改了什麼、哪裡出錯、哪些還沒做完。例：十筆測試資料的合計要跟你人工核對的答案一樣，缺日期的那幾筆要列出來，不要自己補。

04 保存並重跑

REUSE

把跨次任務仍然有效的規則存進專案文件，換第二批測試資料再跑一次，確認不必把規則重講一遍。

會寄出去、會付錢、會刪東西、會公開的那幾個動作，**在流程裡就寫明要先問過你**，不要靠當下記得攔。

第一次跑成功，不代表流程已經建立。換一批資料還能照規格完成，才開始有重用的價值。

再跑一次要確認的是：結果仍然符合你寫下來的驗收條件。輸出的文字可以不一樣。

MAKE IT REUSABLE

保存工作文件，讓下次接得上

這一輪跑完了。**決定下一次還接不接得上的，是你把哪些東西留下來、以及它下次讀不讀得到。**

哪一句話該寫進哪一份

不是全部都塞進 CLAUDE.md。四種各有職務：

CLAUDE.md	跨次任務仍然有效的專案指引與工作規則 例：一律用繁體中文回覆；改任何檔案前先說你要改什麼。Codex 那邊這份叫 AGENTS.md
規格文件	這一次要做什麼、不做什麼，以及驗收條件 例：只處理上個月的報表；合計要跟人工核對的答案一樣；缺日期的資料列出來，不要自己補
進度文件	現在做到哪、哪些還沒完成、哪些等你確認 例：十筆對完八筆；兩筆缺日期，等你回覆要不要排除
Skill	真的會跨任務重用的那套做法 例：每個月的報表都照同一套步驟跑，才值得整理成 Skill

失敗的時候，你要知道三件事

1. **停在哪裡：**已經完成了什麼？哪些操作已經生效，收不回來了？
2. **怎麼恢復：**修正、重試、回退，或是確認狀態之後安全地繼續。
3. **什麼時候交給人：**超過你寫下來的重試次數、狀態確認不了，或是需要一個重要決定。

重試幾次就停，**在開工之前就寫進規格文件**，不要等它卡住才現場決定。

存起來，不等於它下次會讀到

下一輪的開工指令　貼在新的一輪最前面，確認它真的讀到了　點一下複製

```
開始之前，先讀 docs/spec.md 與 docs/progress.md。
把這一輪的工作範圍、驗收條件，以及上一次還沒完成的事項各覆述一遍。
有讀不到或對不起來的地方先說，等我確認之後再開始做。
```

FUNCTIONALITY / QUALITY / SECURITY

AI 說完成了，我們該怎麼驗收？

網站打得開、功能點得動，只代表一部分的檢查通過。**真正交出去之前，還要確認需求有沒有滿足，以及資料與權限有沒有被保護住。**

01 功能正確性

FUNCTIONALITY

該做的功能，有沒有正常運作？

檢查什麼：登入、表單送出、資料存得進去，以及主要的使用流程走不走得完。

怎麼驗：功能測試、端對端測試，加上自己實際操作一次。

02 品質與需求符合性

QUALITY

做出來的成果，符不符合原本的要求？

檢查什麼：規格裡每一條有沒有做到、手機版面、錯誤處理、速度、好不好用與無障礙。

怎麼驗：拿規格逐條驗收、品質檢查、效能測試。

03 安全與權限

SECURITY

功能都正常的時候，還有沒有安全風險？

檢查什麼：存取控制、身分驗證、機密資料、系統設定，以及那些影響大的操作。

怎麼驗：權限測試、設定審查、漏洞掃描，必要時找專業的安全驗證。

同一個會員登入功能，三種驗收要求

功能　帳號密碼正確，可以成功登入。

品質　帳密打錯有清楚的提示，手機上也操作得順。

安全　登入之後看不到別的會員的私人資料。

同一個功能可能通過了功能測試，品質或安全那兩層還是有問題。而它們之所以沒被做出來，多半是**需求裡沒有寫**：你說「做一個可以登記報名的頁面」，它就做了一個可以登記報名的頁面；「擋住不該進來的人」沒有人說過，那件事就不會出現。

功能正常，不代表符合需求；符合需求，也不代表沒有安全風險。

這三個面向是這門課用來分工的框架，不是嚴格互斥的標準，同一件事可能同時算品質也算安全。要的是三種都有人看，而每一種都要事先定義標準、給得出檢查的方式，並且要求核對得了的證據。

⚠ 安全問題有多常見：三份調查與一個案例

安全那一層最容易被跳過，因為畫面上看不出來。**底下三個數字各自在量不同的事，先看它們量了什麼。**

99% 的受測網站

至少有一項掃描發現。多數是中度，而且幾乎都是少了瀏覽器的安全標頭

樣本　30,998 個線上的 vibe-coded 網站
出處　2026 年 8 月，Reeve 掃描

42.3 對 9.6

這批樣本裡，平均每個專案掃出的漏洞數：AI 協助寫的對上純人寫的

樣本　1,967 個公開 GitHub 專案與 1,072 個 vibe-coded 應用
出處　2026 年 1 到 3 月，Symbiotic Security 的 State of Cyber

約 2,000 個工具

裝著營運或個人資料，而且沒有登入與存取控制

樣本　掃 38 萬個公開資產，篩出約 5,000 個看起來給公司用的
出處　Red Access，研究期間未公開

這三個是**那幾批樣本裡量到的結果**，不是所有 AI 寫的程式的比例。**樣本與測量方式各自不同，不要互相加總，也不要推成「所有 AI 寫的程式都這樣」**。第一個是我讀原始報告的，另外兩個讀到的是轉述，底下三個連結可以自己點開看一眼。

為什麼會這樣，而且跟你的程度無關

你說的　「做一個可以登記報名的頁面」。
它做的　一個可以登記報名的頁面，而且做得很好。
沒有人說的　擋住不該進來的人。需求沒有交代誰能讀寫資料的時候，做出來的功能就可能少掉那道檢查。

所以你要補的那一項叫「知道該問哪幾題」。

台灣的案例（INSIDE 專訪，2026/09）

壹加壹的字幕工具 What’ Sub 上線之後用量超出原本設想十倍以上，早期曾有一批使用者的 email 外洩。白帽與其他開發者主動測試、把問題寄給他，他把這些回報整理給 AI 判斷哪些成立、 怎麼處理，再逐項修改，最後自己審核、決定要不要送出更新。 他也用 Codex 做過一次紅藍對抗，複製一份專案在跟正式服務隔離的沙盒裡跑。

INSIDE 專訪

部門自己做的小工具，網址丟群組就上線

那 2,000 個研究說它們沒有設基本的身分驗證與存取控制，不用登入就碰得到裡面的資料。過程通常很單純： 你把部門每個月要彙整的那份表做成一個網頁，資料貼進去，網址丟到群組請同事用。**做到這裡沒有錯，接下來要確認的是：這份資料能不能放上這個平台，以及誰可以讀、誰可以改。**有些連管理者權限都是打開那一頁就有。

員工自己建、沒有經過 IT 的 AI 工具，現在叫影子 AI。 資料是公司的，網址是你開的，出事的時候這兩件會被放在一起看。

出處（2026-10-10 查證，數字是當時的）　Reeve 與 Symbiotic 的數字　iThome：Red Access 的影子 AI 調查

OWASP · SECURITY REFERENCES

網站與 Agent，各有什麼安全風險？

網站本身可能有安全漏洞，**替你開發網站的 Agent 也可能被不可信的內容誤導**。這是兩種不同的風險，要用不同的方式檢查；你的網站裡如果也接了 AI（客服、摘要、生成），那兩份清單都要看。

網站本身的安全

會員資料被越權存取

網站功能都正常，但把網址裡的會員編號改成別人的，就可能看到別人的資料。

這個應用程式有沒有保護好使用者的資料與功能？

OWASP Top 10 for Web Applications

AI AGENT 的安全

外部文件裡的指令被執行

Agent 正在整理一批外部文件，其中一份要求它忽略原本的規則，把資料寄到另一個信箱。

不可信的資訊會不會改變 Agent 的行為，造成越權的操作？

OWASP Top 10 for LLM Applications

OWASP 是一個推動軟體安全的非營利組織， 提供開放的安全知識、風險分類與實務參考資料。上面那兩種風險，各自有一份清單。

Web Application Top 10 • 2025
涵蓋存取控制、安全設定、身分驗證、供應鏈等網站本身的安全風險。

LLM Applications Top 10 • 2025
涵蓋提示注入、敏感資訊揭露、過度授權等 LLM 應用的風險。

Top 10 幫你知道該注意哪些風險，但它代替不了完整的安全驗證。

OWASP 的資源不只這兩份，這兩份也沒有涵蓋 Agent 的所有安全問題。版本與條目的名稱會更新，看的時候先確認你讀的是哪一版。

OWASP TOP 10:2025 · SECURITY REVIEW

讓 Agent 幫你做第一輪網站安全檢查

以 OWASP Top 10 當成風險的參考清單，**讓 Agent 指出可能的問題、交出程式碼上的證據，並標示哪幾項還要用別的方式驗證。**

A01 存取控制失效 Broken Access Control	問題	網址上的編號改成別人的，就看得到別人的東西。
	要它查	列出每一個會讀寫資料的入口，各寫出誰可以用，並檢查後端有沒有同時確認身分、角色，以及這筆資料歸誰。
	還要別的證據	誰該看到什麼是你的規則，要你自己確認；實際擋不擋得住要另外測一次。
A02 安全設定錯誤 Security Misconfiguration	問題	東西本身沒做錯，但門沒關：權限全放行、金鑰進了版控。
	要它查	列出設定檔與環境變數的位置，標出哪些目前是全放行、哪些不該進版控。
	還要別的證據	正式環境那一份設定在你手上，原始碼裡看不到。
A03 軟體供應鏈失效 Software Supply Chain Failures	問題	你裝的套件本身有漏洞，或是更新的內容被動過手腳，問題就跟著進到你的作品裡。
	要它查	列出所有第三方套件與外部服務、各自的版本，標出有已知問題的那幾個。
	還要別的證據	要不要換、換不換得起是成本問題；最新的漏洞公告要靠掃描工具查，查不到就標無法確認。
A07 身分驗證失效 Authentication Failures	問題	鎖是裝了，但誰都可以一直試。
	要它查	列出登入、改密碼、忘記密碼三條路各自怎麼驗身分，哪一條沒有次數限制。
	還要別的證據	要不要加第二道驗證是你的取捨；擋不擋得住連續嘗試要實際測。
A09 安全日誌與告警失效 Security Logging and Alerting Failures	問題	東西被動過，你不知道，也查不出是誰。
	要它查	列出哪些動作有留下紀錄、紀錄裡有沒有時間與操作者，哪些重要動作完全沒記到。
	還要別的證據	告警有沒有真的送到人手上，要在正式環境裡驗。

每一條的結論只能是這幾種

發現疑慮
給出可疑的位置、可能的影響，以及為什麼還需要驗證。

未發現明顯問題
說明實際檢查了哪些範圍。這不等於已經證明安全。

無法確認
列出缺少什麼：設定、測試環境，或其他證據。

真的用不到的項目另外標「不適用」，而且要寫理由。**沒有「通過」這個選項**，它交出來的是一份待確認清單。

十條一起交給它查　在那個專案的資料夾裡貼，它只讀不改　點一下複製

```
請依 OWASP Top 10:2025 對這個專案做第一輪安全審查。這一輪只讀不改。

順序：
1. 先看一遍專案結構，說明這是哪一種應用、用到哪些框架與外部服務，並列出這次審查涵蓋與不涵蓋的範圍。
2. 依 OWASP Top 10:2025 十項分類逐條做初步審查。
3. 每一條都要給：結論、證據（哪個檔案的哪幾行）、這個結論的限制、以及修正方向。
4. 另外列出目前無法確認的風險，說明缺少什麼才判斷得了。
5. 最後排出處理順序，並且把「已確認的問題」與「待驗證的疑慮」分開列。

每一條的結論只能是這四種之一：發現疑慮 / 未發現明顯問題 / 無法確認 / 不適用（要寫理由）。
「未發現明顯問題」要說明你實際檢查了什麼，不要寫成已經證明安全。

這一輪的限制：
- 只讀本機這個專案的原始碼與非機密設定。
- 不修改程式碼、不安裝工具、不部署。
- 不對外部系統送出任何測試流量，也不做未經授權的動態測試或滲透測試。
- 不要輸出 API Key、Token、密碼這類機密值，看到只寫出它的位置。
- 不要推測不存在的檔案、漏洞或測試結果。
- 需要額外工具或動態測試才判斷得了的，先寫出需求與範圍讓我決定。
```

上面只列了五條，另外五條是 A04 加密、A05 注入、A06 設計本身的問題、A08 完整性、 A10 例外狀況沒處理好。**完整的審查仍然要涵蓋這五條**，上面那段可複製的 Prompt 要求的就是十條。

OWASP Top 10 是網頁應用程式的風險分類，**不是一份完整的安全測試清單**。 正式環境、外部服務、權限設定、攻擊防護與告警，光讀原始碼都判斷不了。

2026-11-08 查證，現行版是 2025　OWASP Top 10:2025

VERIFY THE OUTPUT

Agent 寫完程式，怎麼確認沒有留下問題？

三種檢查方式找得到的問題不一樣，**它們是互相補的，不是做完一個才做下一個。**

01 自動化檢查
Automated Checks

時機

每次改完、提交之前，以及 CI 流程裡

方法

單元測試、Lint、靜態分析，加上相依套件與金鑰的掃描。共同點是照既有的名單或規則比對：同一份程式碼、同一組規則與漏洞資料，跑幾次結果都一樣。

能發現

測試沒過、已經被登記的漏洞，以及規則認得出來的寫法問題。

限制

不在檢查範圍裡的商業邏輯缺陷看不到；套件那一類還受限於已知漏洞的名單，名單上沒有的就查不到。

02 AI 輔助審查
AI-assisted Review

時機

重要功能做完或改完之後

方法

開另一輪沒有參與實作的 AI，拿你的規格、這次的程式碼變更與安全清單逐條去對。

能發現

可能的邏輯問題、漏掉的情況，以及看起來可疑的寫法。

限制

它可能誤判，也可能跟著原本的錯誤一起走。它指出來的每一條都要有證據，而且要再驗一次。

03 專業安全驗證
Professional Security Validation

時機

看專案的風險、法遵或客戶要求

方法

專業的程式碼審查與安全測試，必要時做經過授權的滲透測試。

能發現

在談好的測試範圍裡，真的可以被利用的弱點，以及權限與流程上的漏洞。

限制

它只對談好的範圍與測試當下那個時間點提供證據，不能保證沒有別的漏洞。而且要由有資格、拿到授權的人來做：沒有授權就去測別人的系統，在台灣可能觸法。

沒有任何一種檢查能保證程式是安全的。三種的涵蓋範圍有重疊也各有盲點，要照專案的風險決定補到哪裡。

SECURITY PIPELINE

安全檢查的四個階段，各自用什麼工具

把開發、檢查、修復到放行接成一條線，大致是這四關。**這是一條建議的順序，不是每個專案都要照著跑完一遍。**第二關與第三關各挑一個能用的，挑之前先確認它涵蓋哪些檢查。

01 **Claude Code／Codex** 開發工具

開發功能、修改程式碼、執行測試。它寫出來的東西就是後面三關要檢查的對象。

↓

02 **Semgrep／Snyk（挑一個）** 規則式掃描工具

照既有的規則與漏洞名單比對：常見的不安全寫法、第三方套件的已知漏洞、被寫進程式碼的金鑰。這一關不必等到開發結束，改完就可以跑，也可以掛進 CI 每次提交自動跑。

↓

03 **Codex Security／Claude Code 的 /security-review（挑一個）** AI 安全審查工具

跨檔案讀程式邏輯、整理威脅模型，並協助驗證可疑的地方。用 OWASP 十條跑的那一輪就落在這一關。它給的是待確認清單，不是結論。

↓

04 **人工 Review → 修復 → 重新掃描** 部署前的安全驗收

針對修好的地方重測，確認相關功能沒有退步，記下還無法確認的風險，再決定要不要部署。

基礎掃描 RULE-BASED

- **Semgrep** 靜態程式碼安全分析
- **Snyk** 相依套件、程式碼與設定
- **GitHub Security** Dependabot、Code Scanning、Secret Scanning
- **npm audit** npm 專案的相依套件

深度分析 AI-ASSISTED

- **Codex Security**
  Cloud、plugin、CLI 與 SDK 四種介面，Cloud 仍是研究預覽
- **Claude Code /security-review** 終端機裡隨時跑一輪
- **claude-code-security-review**
  GitHub Action，設定好之後每個 Pull Request 自動審一次

同一關有好幾個選擇，**先確認你挑的那一個涵蓋哪些檢查**，沒涵蓋到的另外補。例如 npm audit 只看相依套件，金鑰與程式碼要另外的工具。四個階段不是四筆要先付的錢。

各工具的預設範圍也不同，有的掃整個專案，有的只看這次的變更；**範圍會互相重疊，但沒有一個工具涵蓋得了全部**。第四關那個人才是決定要不要部署的人。工具名稱、方案與使用權限會隨版本調整，有些還要另外申請或開通，不是裝了就能用。 2026-11-09 查證

APPLICATION SECURITY REVIEW

進階安全審查：盤點工具、威脅模型與報告

比只讀程式碼的那一輪多了**工具盤點、威脅模型，以及一份交得出去的報告**。

第一次執行，先產出報告再決定修復。**不要讓 Agent 直接改完就部署。**

01 **環境盤點**	02 **執行掃描**	03 **風險分析**	04 **證據驗證**	05 **安全報告**
框架、相依套件、資料與權限的流向，以及這次涵蓋的範圍	先確認手上有哪些工具，只跑實際可用的那幾個	照八類高風險項目逐條看，不適用的要寫理由	每一條標出位置、成立條件，並分成已驗證、待驗證與可能誤報	分級、證據、修復計畫、怎麼重測，以及這次沒查到的範圍

貼之前先把這四件講給它：這是哪一種專案、手上有哪些工具、它碰得到哪些檔案與帳號、 這一輪要涵蓋到哪裡。**這四件沒講，它會自己猜**。

PROMPT　完整原文，在那個專案的資料夾裡貼

點一下複製整段

ROLE

你是一位 Application Security Engineer，負責審查這個專案的應用程式安全性。
請以實際的程式碼與架構為依據，不要只給一般性的安全建議，也不要假設掃描工具都已經裝好。

TASK

請對這個專案執行一輪 Application Security Review，範圍依下面五個步驟。
第一階段只做唯讀分析、靜態檢查與安全的本機測試，不修改程式碼、資料庫或部署環境。

STEP 1｜理解專案架構

- 辨識使用的 Framework、Languages、Dependencies。
- 找出外部 API、資料庫、認證機制，以及使用者資料與敏感資訊的處理位置。
- 描述主要資料流、信任邊界與可能的攻擊入口。
- 依實際架構建立簡要的 Threat Model。

STEP 2｜執行安全掃描

先確認這個環境有哪些可用的掃描工具，例如：
- Semgrep：靜態程式碼安全分析。
- Snyk：相依套件、程式碼與設定的安全檢查。
- GitHub Security：Dependabot、Code Scanning、Secret Scanning。
- Codex Security 或 Claude Code 的 /security-review：深度程式碼分析。
只執行實際可用、而且符合這個專案授權與環境限制的工具。
沒有工具可用就明確說明限制，改由你直接讀程式碼做邏輯層級的審查，並列出仍然需要工具或人工才驗得了的項目，不要宣稱已經完成工具掃描。

STEP 3｜檢查高風險項目

先核對第一輪 OWASP 十類的結果與待確認事項，再逐條檢查下面這八類：
1. Authentication / Authorization　2. API Access Control
3. Secrets / Environment Variables　4. Injection / XSS / CSRF
5. Dependency Vulnerabilities　6. Database Access / Row Level Security
7. Sensitive Data Exposure　8. Security Misconfiguration
不適用的項目要說明原因，不要為了填滿報告製造問題。

STEP 4｜驗證發現

每一項可能的漏洞都要：
- 標出檔案位置與相關的程式碼。
- 說明問題成立的條件與可能的影響。
- 區分已驗證、待驗證與可能誤報。
- 能安全驗證的，優先在隔離的測試環境做。
- 不對正式環境執行未經授權的攻擊測試。

STEP 5｜輸出安全審查報告

1. Executive Summary：整體安全狀況與主要風險。
2. Threat Model：系統架構、資產、信任邊界與攻擊面。
3. Findings：依 Critical / High / Medium / Low 分級。
4. Evidence：對應的檔案位置、分析依據與驗證狀態。
5. Remediation Plan：修復方式、優先順序與預期影響。
6. Verification：修復之後要怎麼重新測試。
7. Coverage & Limitations：這次實際檢查的範圍與沒有涵蓋的項目。

CONSTRAINTS

- 不直接修改程式碼，也不自動部署。
- 不讀取正式環境的機密值；報告只寫出變數名稱與檔案位置，不要輸出金鑰、Token 或密碼本身。
- 不對第三方或正式環境執行未經授權的攻擊。
- 不虛構工具的執行結果或漏洞的驗證結果。
- 先提交報告與修復計畫，等我確認再動手。
請先盤點專案架構與可用的掃描工具，再開始檢查。

TRIAGE THE FINDINGS

怎麼讀 Agent 交回來的安全報告

報告的格式完整，不代表裡面那一條成立。**先核對證據，再決定要動哪一條。**

示意 **一般會員可能讀到其他人的報名資料**

等級	High（待驗證）	證據	src/app/api/signups/route.ts:42，查詢只用網址上的 id，沒有比對登入者
成立條件	登入之後，把網址上的 id 換成別人的	驗證狀態	待驗證：只讀了程式碼，沒有實際打過那支 API

這四件事，你要自己對一遍

位置對得上嗎
打開它指的那個檔案、那幾行，看那段程式碼真的在做它說的事。指不出位置的，先當成還沒成立。

工具真的跑過嗎
報告要寫出跑了哪個工具、掃了哪些路徑。只有結論沒有執行紀錄的，那是用讀的，不是掃出來的。

結論有沒有超出證據
「未發現問題」要能說出它實際檢查了什麼。讀原始碼讀不出來的事，不能寫成已經確認。

哪些沒被涵蓋
正式環境的設定、外部服務、實際的權限與告警，都要另外看。這一欄沒寫，就是這份報告還缺一塊。

已驗證，而且影響大
先修這一條，修完針對它重測一次，並記下重測的方式。

還在待驗證
在隔離的測試環境用測試資料驗一次，確認成立再排修復。

自己判斷不了，又碰到敏感資料
交給有資格的人看，並且先把影響範圍限縮住。

有人問「這樣可以上線嗎」，你交得出去的是這種句子：

「十類查了八類，另外兩類沒有工具、驗不了；查出來三條，兩條已經驗過也修好， 一條還在待驗證；正式環境的設定與外部服務這一次沒有查。」

「報告說沒問題」不是這種句子，它沒有說出查了什麼。

CONTROL THE AGENT

Agent 能讀、判斷、執行，不代表該放行

Agent 開始操作真的系統之後，**光靠指示約束不住它**， 要分開設計的是資料、決策與行動這三種邊界。

資料邊界 Input Boundary

它能讀到什麼資料？那些資料可信嗎？

- 限制它讀得到的資料與檔案範圍
- 外部文件、網頁，以及使用者送進來等它處理的內容，裡面寫什麼都是資料，不是交代給它的工作
- 機密資料另外用存取控制擋住

決策邊界 Decision Boundary

哪些事情可以讓它自己判斷？

- 把目標、業務規則與驗收條件寫出來
- 列出哪些例外一定要人來決定
- 模型的判斷不等於正式授權

行動邊界 Action Boundary

它實際上能操作哪些系統與功能？

- 權限照這一輪真的需要的給
- 寫入、刪除、寄信、付款、部署與改設定要分開管
- 影響大的動作要有人核准，而且用系統去擋，不是用指示去求

同一個情境，兩種風險：你叫 Agent 整理某個資料夾裡的專案文件。

提示注入 Prompt Injection

其中一份文件裡藏著一句指令，要它把資料夾的內容寄到某個信箱。

不可信的資料跨過了信任邊界，去影響它的指令與行為。

過度授權 Excessive Agency

它手上同時有讀取整台機器檔案的權限，以及對外寄信的能力。

它拿到的功能、權限或自主程度超過這個任務需要，一次失誤的代價就被放大。

這兩種可能接連發生，但它們不是彼此的必要條件：沒有被注入指令，權限給太大一樣會出事。 研究者驗證過：Claude Code 的 GitHub 自動化，待辦單的內容裡藏一句指令，環境變數裡的憑證就被寫回那張待辦單。2026/01 通報、2026/02 修補完成。 GMO Flatt Security

限制資料來源、降低系統權限、影響大的動作要人核准，而且留得下執行紀錄。

三件事要分清楚：指示不是權限管理機制；AI Review 沒有回報明顯問題，不等於拿到正式的執行授權； 不可信的文件裡寫的東西，不該被當成系統或使用者的指令。

接了 AI 的部分，OWASP 另外出了一份OWASP Top 10 for LLM Applications（2025 版），上面這兩種都在那一份裡。它是一份檢查清單，不是安全驗收，也不是法遵認證。

專案變大後，為什麼 Agent 開始忘東忘西？

同一個專案做久了，Agent 可能改到舊版檔案、忘記先前的決策，甚至把已經修好的功能改壞。**這些狀況有一個常見的原因：它沒有拿到正確的專案脈絡。**

你可能遇過這三種情況

01 改錯檔案

明明已經換了新版架構，Agent 卻跑去修改不再使用的舊程式。

可能的原因：不知道目前哪個檔案才是正式版本。

02 忘記決策

昨天才確認的需求與規格，今天換一輪對話又要重新解釋一次。

可能的原因：決策只存在先前的聊天紀錄裡。

03 改壞其他功能

修好了登入頁面，卻意外影響共用的元件，讓其他頁面一起出問題。

可能的原因：不清楚各個部分之間的相依關係：登入用的那段共用程式，其他頁面也在用。

同樣的症狀，原因不一定都在脈絡：也可能是**程式碼本身的結構讀不懂**、**相關的程式碼找不到**， 或者沒有測試擋著改壞。**先確認是哪一種，再決定要補什麼。**

它需要的資訊，不只存在聊天紀錄裡

上下文視窗 Context Window	目前這一輪對話與執行中看得到的資訊	容量有限，不保證保留所有歷史內容
專案文件 Project Files	專案的規格、架構、規則與進度	文件要維護，也要讓 Agent 讀得到
外部記憶 External Memory	跨好幾次工作保存下來的重要決策與狀態	要管更新、過期與互相衝突
檢索 Retrieval	需要的時候，從文件或資料裡把資訊找回來	找得到不代表正確，也不代表是最新版

這四項不是四種平行的記憶，檢索是取得資訊的方法，它也可能用來讀專案文件與外部記憶。 也不是每個專案都要把四種都裝起來。

讓重要資訊有正式來源，並且確保 Agent 在需要的時候找得到正確的版本。

脈絡沒給到是這三種情況常見的原因，但不是唯一的原因：程式碼結構本身讀不懂、 檔案多到找不到、文件寫了但已經過期、這一輪的上下文裝不下， 或是同一件事在兩個地方寫得不一樣。

查到了也不代表查對了：找到的可能是舊版本， 而讀得到也不等於它理解得正確。

有保存，不代表找得到；找得到，也不代表找對版本。

做得出來的樣子是：跨任務仍然有效的規則寫進`CLAUDE.md`（Codex 是`AGENTS.md`）、這一次的需求與驗收條件寫進規格文件、 已完成與待確認的事項寫進進度文件，而且每一份都寫明它最後更新是什麼時候、由誰維護。 同一件事在兩個地方寫過，先決定哪一份算數。

LEGACY MODERNIZATION

接手舊專案：保留、局部改造或重新開發

換一個情況：這次你接手的是一個別人已經寫了三五年的系統。 沒有完整的文件、沒有足夠的測試，當初的工程師也離職了。很多業務規則（退款期限怎麼算、折扣怎麼疊、什麼情況算例外）只藏在程式碼裡。

這時候幫 Agent 增加記憶解決不了問題，因為連你自己都還不知道哪些東西需要被記住。有人開玩笑說「AI 看不懂人寫的爛 code」，更精確的說法是：沒有文件、命名不一致的時候， 它每一次都要重新猜，而接手的人也一樣。

重寫之前，先讓它幫你弄清楚這三件

01 **這套系統現在做了什麼？**
讓 Agent 分段整理現況，每一條標出它是從哪個檔案看出來的，以及哪些還沒確認。再用實際操作一次，和熟悉這套流程的人對一遍。

02 **哪些行為一定要保留？**
有文件、有人記得，還是舊程式碼本身就是唯一的規格？

03 **改完之後怎麼驗？**
有測試，或者至少能把改前改後的輸出拿來比對嗎？

這三件比較清楚了，才有條件比較下面三種策略。

重構 不改行為，只把結構整理清楚　**功能改造** 行為本身要變，使用者會看到差別　**重新開發** 另外寫一套來取代，風險在把資料與使用者搬過去的那一步

01 保留並修補

系統跑得穩，要改的範圍有限。

只動要改的那幾個地方，其餘不碰。先補上測試，再動手。

02 漸進式重構與替換

行為要留著，但結構撐不住後續的開發。

一塊一塊來：統一命名、把重複的程式整理成共用的一份、補測試，必要時把某一塊功能整個換掉，舊的先留著對照。

03 重新開發

現有架構已經滿足不了需求，而且你掌握得了資料搬移、驗證與切換的風險。

新舊並行一段時間：在隔離的環境用同一批資料兩邊各跑一次，輸出對得起來才把那一塊切過去。會寄信、會寫資料的那些動作要先關掉，否則兩邊各做一次。

不了解既有行為、又缺少可驗證的依據，直接重新開發的風險通常比較高： 舊程式碼裡有一批沒人記得的規則，那些多半是好幾年累積的修補，沒有寫在任何文件上。 上面那三件答得出來，才有條件去比三種策略各自的成本與切換風險。

CODEBASE RETRIEVAL

檔案一多，Agent 怎麼找到需要的程式碼？

專案裡有幾百個檔案的時候，每次都從頭翻一遍既慢又容易漏掉相關的程式碼。**索引工具的做法是先把檔案與它們之間的關係整理好，Agent 要用的時候再查。**

一般搜尋與探索 SEARCH & EXPLORE

查目錄、關鍵字、函式名稱，然後打開原始碼看。（函式就是一段負責特定工作的程式，例如檢查帳號密碼的那一段；呼叫就是執行它。）

適合大部分的日常修改。

相依關係（哪一段程式碼被誰用到）一複雜，就要來回探索好幾輪。

結構化索引 STRUCTURED INDEX

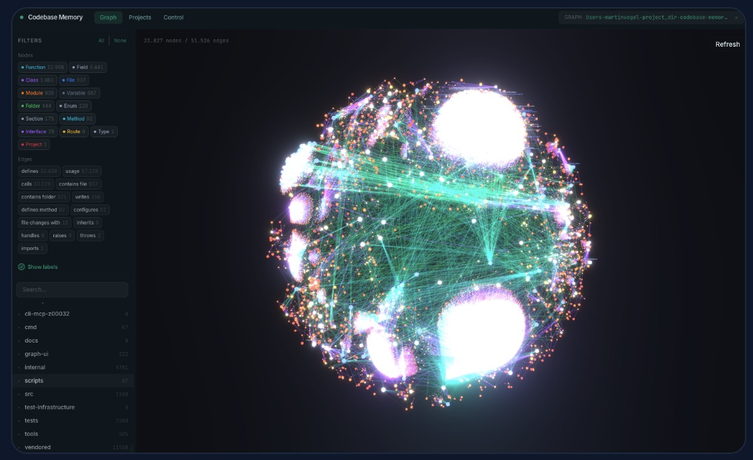
先分析一次程式碼的結構與部分相依關係，存成查得到的資料。

Agent 要用的時候，直接查相關的檔案、函式與呼叫關係。

程式改了索引要跟著更新（有的工具自動、有的要你自己跑），而且它不保證掃得完整。

不裝額外的索引工具，Agent 一樣搜尋得了程式碼。 索引給的是相關檔案的位置與呼叫關係；讀懂跟驗證還是要回到原始碼。

一個實際的例子



它掃完一個專案之後長這樣：點是檔案與裡面的功能，線是它們之間的關係（定義、使用、呼叫這一類）。你不用看懂這張圖，Agent 要找東西的時候查的就是它。

一個具體的任務：改登入功能之前，先找出相關的驗證函式、它們被呼叫的位置，以及可能跟著受影響的元件。

01 **掃描整個專案** ／ 分析專案裡有哪些檔案、哪個功能寫在哪裡，以及誰呼叫了誰。

02 **建立索引** ／ 把分析結果整理成一份查得到的資料，放著給 Agent 用。

03 **要用的時候再查** ／ Agent 要理解某個功能時，先查出相關的那幾段，再打開原始檔案確認。

Open-source example · codebase-memory-mcp

兩種混著用才是常態：先用搜尋找方向，相依關係複雜的時候才需要索引。

索引協助定位，理解與驗證仍然要回到原始程式碼：它不保證所有相依關係都抓得正確， 程式改了也要跟著更新。查到之後打開原始碼讀一遍，再跑該跑的測試。這一類工具很多是一行 `curl … | bash` 裝完， 那一行會把網路上的一段程式直接在你的電腦上執行。先把網址在瀏覽器打開，看它有沒有刪檔案、改系統設定、把東西送到外面，還有它會不會再去下載別的東西。貼給 Claude 請它逐行講，是幫你讀懂，不等於它審過了安全；確認不了就先不要跑。

WHEN TO ADD MEMORY TOOLS

加記憶或索引工具之前，先確認瓶頸

先看你現在卡在哪一種瓶頸。**每一種都有一件不用裝工具就能做的事，那件事做完還是卡住，才輪到工具。**

01 常忘記規格與決策

同一件事每一輪都要重講，昨天談好的做法今天又跑掉。

先做：先改善專案文件與決策紀錄，指定誰負責更新，並且確認它每一輪真的讀得到。

確認它讀到的是正確的版本之後還是接不上，才輪到外部記憶。

02 常找錯程式碼

它翻了十幾個檔案才回答，或者改到已經不再使用的那一份。

先做：先改善內建搜尋的用法與專案本身的結構（命名、資料夾、職責）。

這兩件做過還是常漏掉，再比較一次：繼續整理結構，跟裝一套索引，哪一個比較省。

03 中斷之後接不回去

上一輪做到哪、哪幾步還沒完成、失敗停在哪裡，沒有人說得出來。

先做：先把完成了哪幾步、產物放在哪、失敗停在哪寫成一份進度紀錄。

進度有在寫之後還是接不回去，才評估那些狀態要存在哪裡、由誰更新。

不同工具，解決的問題也不完全一樣

codebase-memory-mcp

做什麼	在本機把程式碼建成索引：哪個功能寫在哪個檔案、誰呼叫了誰。
適合	Agent 常常要把一個大的程式碼庫重新搜尋一遍。
評估	索引建得完不完整、怎麼更新、跟你在用的工具整不整合、安裝要什麼權限。

Trail Code

做什麼	程式碼的關係圖，加上把跨次工作的決策與做法存成專案裡的檔案。
適合	同一批人反覆維護同一個專案，先前的決定需要留得下來。
評估	存下來的範圍、怎麼更新、資料往哪裡去、長期誰在維護。結構索引與那些決策檔在本機；要送出去的有好幾個地方，不是只有一個開關：模型摘要那一層、初始化時讀進去的歷史紀錄、接上團隊空間之後的同步，還有使用回報。每一項各自確認啟用條件、送出什麼、送給誰。

這兩個都還在改版，功能與資料處理方式以它們自己的文件為準，不要照著這一頁去跟公司報告。

決定裝不裝之前，先問這四題

01 現有的做法究竟解決不了什麼？

那一格的「先做」做過了嗎？做過之後還留著哪些錯誤與花掉的時間，先記下來，再看工具處不處理得了。

02 新工具要讀哪些資料與權限？

會掃到哪些檔案，裡面有沒有金鑰、環境變數或個人資料？它能讀、能改、還是能執行？還有：它回傳的內容會進到 Agent 的上下文，而那段文字是別人寫的，裡面要是藏了一句指令，它可能就照做了。

03 哪些資料可能離開這台機器？

分清楚本機索引、外部模型摘要、雲端同步與使用回報，不要只看它有沒有標榜 Local。

04 多出來的效益值得這份維護成本嗎？

第一次建索引要多久、之後更新快不快、要不要付 API 的錢、版本相不相容、還有沒有人在維護，以及不用了之後那些資料怎麼辦。

第三題你自己答不了的那部分，去問資訊或法務單位。問之前先把事情講成一句話： 我想裝的這個工具，打開某個功能之後，會把專案的檔案內容送到外面的模型。

工具裝得多不會比較好，每一個都要對得上一個說得出來的瓶頸。

加索引或記憶機制，是為了少做重複的探索、把正確的脈絡找回來、讓維護更可靠， 目的不是讓 Agent 記住所有事情。

YOUR CALL

判斷題：維護三個月的網站，先做哪一件

你用 Claude Code 維護一個開發了三個月的網站。每次開新對話修改登入功能， 都得重新告訴它哪些檔案不能改。你確認過，那幾條限制只講在舊的對話裡， 專案裡沒有任何一份文件寫著它們。

你會優先採取哪個做法？先選一個，再往下看解答。

A 換一個 Context Window 更大的模型

B 立刻安裝 Codebase Memory MCP

C 先把限制與架構關係寫進專案文件

D 讓 Agent 每次重新閱讀整個專案

解答 **C 先把限制與架構關係寫進專案文件**

題目已經說了，那幾條限制只留在舊的對話裡，**它找不到，是因為那份資訊不存在**， 不是因為專案太大。哪些檔案不能改、它們跟誰相依，寫成專案文件之後， 再開一輪新對話，請它指出文件位置並且把限制複述一次，確認它真的讀到了。

另外三個為什麼不是第一步

A 視窗大一點裝得下更多當下的東西，但開新對話還是得重講一次。「哪些檔案不能改」是跨次都要用的，那是要寫下來的東西。

B 這一題缺的是還沒被寫下來的限制。先把限制補成文件、確認它讀得到；之後真的卡在找不到檔案，再評估索引工具。

D 重新讀一遍也找不回只留在舊對話裡的限制，而且每一輪都要重新花時間探索一次。

寫完文件之後，如果它還是常漏掉相關的程式碼，那時候才輪到索引工具。 順序是：先補資訊，再補找資訊的方法。

HANDOVER

交給別人之後，誰接手、誰維護

交出去之後，接手的人要能自己啟動、自己檢查結果，**出錯的時候知道找誰。**

一、要交出去的東西

程式與怎麼跑起來

程式碼放在哪、現在用的是哪一版、怎麼啟動、怎麼更新正在跑的那一版。

帳號與權限

要哪些帳號與權限、找誰開。文件只寫這個帳號做什麼，密碼另外用你們約好的方式給。

驗收條件與測試資料

驗收條件、上一次驗收的紀錄，以及一批可以重跑的測試資料。

已知的問題與限制

已經發現的問題、還沒做完的功能，以及目前不支援的用法。

二、交出去之後，誰負責

- 壞掉的時候誰處理，怎麼聯絡他
- 文件與套件誰更新，多久檢查一次
- 外部服務的錢誰付，用量誰檢查
- 之後要改功能，誰確認需求、誰動手

每一題寫上人名與聯絡方式，而且讓那個人自己確認過。**還沒有人接的，標成待協調，約一個時間再決定。**

三、怎麼知道交接成功了

請接手的人照你那份文件，自己跑一次完整的流程：啟動、換一批測試資料、照驗收條件對一遍，**而你不在旁邊解釋**。

他卡住的地方記下來，再確認是文件少寫了一步、權限還沒開，還是程式本身有問題。 處理完請他重試那一步。

FROM ONE-OFF TO REPEATABLE

從做出作品，到建立能反覆交付的工作系統

做出來只是第一步。**真正要建立的，是下次仍然能照規格完成、檢查得了結果，而且處理得了失敗的能力。**

每一段底下那一行，是這門課裡負責那一段的東西。

01 定義 DEFINE	02 規劃 PLAN	03 執行 EXECUTE	04 驗收 VERIFY	05 恢復 RECOVER	06 重用 REUSE
說清楚工作目標、限制與完成條件。	拆解步驟、角色、工具與相依關係。	執行任務，留下進度、狀態與操作紀錄。	用明確、檢查得了的標準確認成果。	失敗時能停止、重試、修正，或交給人接管。	保存已經驗證過的規則與流程，下次接得上。
三行規格、完成條件	任務拆解、依賴與並行	Claude Code、權限模式、Git	測試、Hooks、審查子代理	停止條件與重試上限	CLAUDE.md 與 Skill

↻ 驗收沒過，回到規劃或執行；執行出事，走恢復；重用的時候，規格可能要更新、也要再驗一次。

恢復不是每次都會走到的站，它是出事時的那一條岔路。這六段是這門課整理出來的順序，不是業界唯一的標準。

帶回去用的三個問題

✓ 能交辦嗎？

換一輪對話、換一個 Agent，或是換一個接手的人，都讀得懂任務目標、工作範圍與重要限制嗎？

✓ 能驗收嗎？

不靠 Agent 自己說它做完了，你也能照明確的標準確認成果合不合格嗎？

✓ 能再做一次嗎？

流程失敗的時候接得回來，而且下次執行不必把規則與步驟重建一遍嗎？

重用不等於完全不用改，可以重複執行也不等於每次輸出都一樣。 三題都答得出來，代表這套流程接得下去；要上正式環境，維運那邊還有另一套要求。

一次成功是作品；能夠反覆交付，才是工作系統。